LEARN BEING FORWARD

Programming — C++ Programming

C++ fundamentals & OOP

COMPLETE LEARNING GUIDE

What You'll Learn

C++ began as 'C with Classes' and still compiles the overwhelming majority of valid C, but it layers on an enormous set of features — references, function overloading, namespaces, a safer and more expressive I/O system, and ultimately full object-oriented programming — that change idiomatic style so much that experienced C++ code often looks nothing like the C it's built on. This sub-module bridges that gap: it covers exactly what C++ adds on top of C syntax and semantics (references, overloading, default arguments, namespaces, streams) and previews the class-based OOP model that the next sub-module covers in full depth. Getting comfortable with these additions first is what makes the jump into classes, inheritance, and templates later feel like a natural extension rather than an entirely new language.

1. From C to C++: What's New at the Language Level

C++ adds true references, function and operator overloading, default function arguments, namespaces to avoid name collisions between libraries, a real bool type (built in since the start, unlike C's later stdbool.h addition), and the iostream-based cin/cout for input/output instead of printf/scanf's format-string approach. It also tightens some of C's looser rules — C++ requires every function to be declared/defined before use with no implicit-int fallback, and enforces stricter type checking around void* conversions — trading a small amount of C's flexibility for a large amount of compile-time safety.

- bool, true, and false are built-in keywords in C++ (C only gained them via <stdbool.h> in C99, and even then as macros for _Bool).
- C++ requires an explicit cast when converting void* to another pointer type; C allows it implicitly.

```cpp
#include <iostream>

int main() {
    bool isReady = true;
    std::cout << "Ready: " << isReady << "\n"; // prints 1 (bool prints as 0/1 by default)
    return 0;
}
```

2. References vs. Pointers

A reference (int &r = x;) is an alias for an existing variable — once bound, it can never be rebound to refer to anything else, and it can never be null, unlike a pointer which can be reassigned freely and can be null. References are overwhelmingly used as function parameters to avoid copying large objects while still allowing the function to read (const int&) or modify (int&) the caller's actual argument, which is more readable at the call site than pointer syntax (no & needed when calling, no * needed to use it inside the function). A reference must be initialized at the moment it's declared — there's no such thing as an uninitialized, 'null' reference waiting to be bound later.

- A reference parameter (void f(int &x)) lets a function modify the caller's variable with plain function-call syntax — no &x at the call site, no *x inside the function.
- Never return a reference to a local variable — it refers to stack memory that no longer exists once the function returns, producing a dangling reference.

```cpp
#include <iostream>

void increment(int &x) { // reference parameter — modifies the caller's variable directly
    x++;
}

int main() {
    int value = 5;
    increment(value); // no & needed at the call site
    std::cout << value << "\n"; // 6
    return 0;
}
```

3. Function Overloading and Default Arguments

C++ allows multiple functions to share the same name as long as their parameter lists differ in type or count — the compiler picks the best match at compile time based on the arguments supplied, a feature C does not have at all (C requires every function name to be globally unique). Default arguments let a parameter be omitted at the call site, using a specified fallback value instead — useful for optional configuration parameters without needing a whole family of overloads just to vary one setting. Default arguments must be specified from right to left (once one parameter has a default, every parameter after it must also have one).

```cpp
#include <iostream>

int add(int a, int b) { return a + b; }
double add(double a, double b) { return a + b; } // overload: different parameter types
void greet(const std::string &name, const std::string &greeting = "Hello") {
    std::cout << greeting << ", " << name << "!\n";
}

int main() {
    std::cout << add(2, 3) << "\n";       // 5    — calls the int version
    std::cout << add(2.5, 3.5) << "\n";   // 6    — calls the double version
    greet("Asha");                        // uses the default greeting "Hello"
    greet("Ravi", "Welcome");              // overrides the default
    return 0;
}
```

4. Classes 101: Grouping Data and Behavior

A C++ class bundles data members (variables) together with member functions (behavior that operates on that data) into a single type — the central idea of encapsulation, where an object's internal state is manipulated only through a defined set of functions rather than accessed arbitrarily from anywhere in the program. Where a C struct is purely a bundle of data, a C++ class (and, in fact, a C++ struct too, which supports exactly the same features) can additionally hide data behind private access and expose only a controlled public interface. The next sub-module goes deep on constructors, inheritance, and polymorphism; this is just enough to make sense of iostream's std::cout being an object, and std::string being a class, not a primitive.

```cpp
#include <iostream>
#include <string>

class Greeter {
public:
    Greeter(const std::string &name) : name_(name) {} // constructor
    void sayHello() const { std::cout << "Hello, " << name_ << "!\n"; }
private:
    std::string name_; // hidden from outside the class
};

int main() {
    Greeter g("Priya");
    g.sayHello(); // Hello, Priya!
    return 0;
}
```

5. Namespaces and the std Namespace

A namespace groups related names (classes, functions, variables) under a common prefix to prevent collisions between identically-named things from different libraries — the entire standard library lives inside namespace std, which is why std::cout and std::string need the std:: qualifier (or a using declaration) to be found. using namespace std; imports every name from std unqualified, which is convenient in small example programs but considered poor practice in headers or larger codebases, since it reintroduces exactly the name-collision risk namespaces exist to prevent; a targeted using std::cout; (importing only what's actually needed) is the safer middle ground.

- Never write 'using namespace std;' at the top of a header file — it pollutes every file that includes that header with the entire std namespace.
- std::cout, std::cin, std::string, std::vector — every standard-library name is namespaced under std to avoid colliding with your own code's names.

6. Input/Output Streams: cin and cout

C++ streams replace printf/scanf's format-string approach with a type-safe, chainable operator-based syntax: << inserts a value into an output stream (std::cout << x), and >> extracts a value from an input stream into a variable (std::cin >> x) — the compiler picks the correct formatting/parsing behavior for each type automatically via operator overloading, eliminating the entire category of printf format-specifier mismatches (like using %d for a double) that plague C. Chaining multiple << or >> calls in one statement is idiomatic and reads left to right in the order values are inserted or extracted.

```cpp
#include <iostream>

int main() {
    int age;
    std::string name;

    std::cout << "Enter your name: ";
    std::cin >> name;
    std::cout << "Enter your age: ";
    std::cin >> age;

    std::cout << name << " is " << age << " years old.\n"; // chained <<
    return 0;
}
```

7. RAII: Resource Acquisition Is Initialization

RAII is the single most important idiom in idiomatic C++: tie a resource's lifetime (memory, a file handle, a mutex lock) to an object's lifetime, acquiring the resource in the constructor and releasing it automatically in the destructor, so the resource is guaranteed to be released when the object goes out of scope — even if an exception is thrown partway through a function, something C's manual malloc/free discipline has no automatic answer for. std::string and std::vector are everyday examples: they manage their own internal heap

memory via RAII, which is exactly why C++ code using them almost never calls new or delete directly. Memory management & smart pointers (a later sub-module) extends this same idea to general dynamic allocation.

- An object's destructor runs automatically when it goes out of scope — this is the mechanism RAII relies on, and it runs even if an exception unwinds the stack.
- std::string, std::vector, std::ifstream, and std::lock_guard are all everyday standard-library RAII types — none of them require you to manually release anything.

Common Pitfalls

- Writing 'using namespace std;' inside a header file, polluting every translation unit that includes it.
- Returning a reference to a local (stack) variable from a function, producing a dangling reference once the function returns.
- Forgetting that a reference must be initialized when declared and can never be rebound to a different variable afterward.
- Expecting C++ function overloading to work based on return type alone — overload resolution only considers parameter types.
- Mixing scanf/printf-style formatting habits into cin/cout code, missing the type-safety benefit streams actually provide.
- Manually managing memory with new/delete out of old C habits instead of relying on RAII types like std::string and std::vector.

Key Takeaways

- References are non-null, non-rebindable aliases — prefer them over pointers for function parameters that don't need to be optional or reseatable.
- Function overloading resolves by parameter types at compile time; C has no equivalent feature.
- Namespaces (and the std:: prefix) prevent name collisions — avoid 'using namespace std;' in headers.
- cin/cout use type-safe operator overloading (<<, >>), eliminating printf/scanf's format-specifier mismatch bugs.
- RAII ties resource lifetime to object lifetime via constructors/destructors — it's the foundation nearly all idiomatic modern C++ resource management builds on.

Learn More

cppreference — C++ Language Reference
https://en.cppreference.com/w/cpp/language

learncpp.com — Introduction to C++
https://www.learncpp.com/